

1. USER SERVICE

Test Case ID	Test Objective	Pre-condition	Test Steps & Input Data	Expected Result & Post-condition
TC_USER_01	Verify successful new user registration	1. User Service running on port 5004. 2. Email does not exist in the system.	Step: Send POST request to /api/auth/register with valid JSON. Input: { "email": "test@example.com", "password": "securepassword123" }	Status: 201 CreatedResponse: {"message": "User registered successfully"} Post: New user saved in DB with hashed password.
TC_USER_02	Verify registration with existing email	1. User Service running. 2. Email already registered.	Step: Send POST request to /api/auth/register with existing email. Input: { "email": "existing@example.com", "password": "password123" }	Status: 400 Bad RequestResponse: {"error": "Email already exists"} Post: No new user created.
TC_USER_03	Verify successful login	1. User registered. 2. User Service running.	Step: Send POST request to /api/auth/login with correct credentials. Input: { "username": "test@example.com", "password": "securepassword123" }	Status: 200 OKResponse: JSON containing token, role, and email. Post: Token saved in tokens table; User authenticated.
TC_USER_04	Verify login with	1. User Service	Step: Send POST	Status: 401

	invalid credentials	running.	request to /api/auth/login with wrong password. Input: { "username": "test@example.com", "password": "wrongpassword" }	UnauthorizedResponse: {"error": "Invalid credentials"} Post: No token created; User not authenticated.
--	---------------------	----------	---	---

2. MOVIE SERVICE

Test Case ID	Test Objective	Pre-condition	Test Steps & Input Data	Expected Result & Post-condition
TC_MOVIE_01	Verify creating a new movie	1. Movie Service running on port 5001. 2. User has admin rights.	Step: Send POST request to /api/movies. Input: { "title": "Inception", "genre": "Sci-Fi", "duration": 148, "release_date": "2010-07-16" }	Status: 201 CreatedResponse: Movie info including generated ID. Post: New movie added to database.
TC_MOVIE_02	Verify retrieving all movies	1. Movie Service running. 2. DB has at least one movie.	Step: Send GET request to /api/movies. Input: None.	Status: 200 OKResponse: Array of all movies. Post: Database unchanged.
TC_MOVIE_03	Verify retrieving movie details by ID	1. Movie Service running. 2. movie_id exists.running.	Step: Send GET request to /api/movies/{movie_id}. Input: URL param movie_id (e.g., "1").	Status: 200 OKResponse: Detailed movie information. Post: Database unchanged.
TC_MOVIE_04	Verify retrieving non-existent movie	1. Movie Service running. 2. movie_id does not exist.	Step: Send GET request to /api/movies/{movie_id}. Input: URL param movie_id = "99999".	Status: 404 Not FoundResponse: {"error": "Movie not found"} Post: Database unchanged.

TC_MOVIE_05	Verify updating movie details	1. Movie Service running. 2. movie_id exists. 3. Admin rights.	Step: Send PUT request to /api/movies/{movie_id}. Input: { "title": "Inception Updated", "genre": "Sci-Fi/Thriller", "duration": 148, "release_date": "2010-07-16" }	Status: 200 OKResponse: Updated movie info. Post: Database updated.
TC_MOVIE_06	Verify deleting a movie	1. Movie Service running. 2. movie_id exists. 3. Admin rights.	Step: Send DELETE request to /api/movies/{movie_id}. Input: URL param movie_id.	Status: 200 OKResponse: Deletion confirmation. Post: Movie removed from database.
TC_MOVIE_07	Verify searching movies	1. Movie Service running. 2. DB contains movies.	Step: Send GET request to /api/movies?query={keyword}. Input: Query param query = "Inception".	Status: 200 OKResponse: List of movies matching keyword. Post: Database unchanged.

3. SHOWTIME SERVICE

Test Case ID	Test Objective	Pre-condition	Test Steps & Input Data	Expected Result & Post-condition
TC_SHOWTIM E_01	Verify creating a new showtime	1. Movie Service running. 2. movie_id exists. 3. Admin rights.	Step: Send POST request to /api/showtimes. Input: { "movie_id": "1", "start_time": "2025-12-01 18:00", "end_time": "2025-12-01 20:20", "price": 75000, "room_id": "R1" }	Status: 201 CreatedResponse: Showtime info including ID. Post: Showtime created; Seats initialized.
TC_SHOWTIM E_02	Verify retrieving showtimes	1. Movie Service running. 2. DB has showtimes.	Step: Send GET request to /api/showtimes (optional movie_id filter). Input: Query param movie_id = "1".	Status: 200 OKResponse: List of showtimes (filtered if param provided). Post: Database unchanged.
TC_SHOWTIM E_03	Verify retrieving showtime details	1. Movie Service running. 2. showtime_id exists.	Step: Send GET request to /api/showtimes/{showtime_id}. Input: URL param showtime_id.	Status: 200 OKResponse: Detailed showtime info. Post: Database unchanged.
TC_SHOWTIM	Verify updating	1. Movie Service	Step: Send PUT	Status: 200

E_04	showtime	running. 2. showtime_id exists. 3. Admin rights.	request to /api/showtimes/{showtime_id}. Input: { "start_time": "2025-12-01 19:00", "end_time": "2025-12-01 21:20", "price": 80000, "room_id": "R2" }	OKResponse: Updated showtime info. Post: Showtime updated in DB.
TC_SHOWTIM E_05	Verify deleting showtime	1. Movie Service running. 2. showtime_id exists. 3. Admin rights.	Step: Send DELETE request to /api/showtimes/{showtime_id}. Input: URL param showtime_id.	Status: 200 OKResponse: Deletion confirmation. Post: Showtime removed; Related bookings cancelled.

4. BOOKING SERVICE

Test Case ID	Test Objective	Pre-condition	Test Steps & Input Data	Expected Result & Post-condition
TC_BOOKING_01	Verify booking a single seat	1. Booking Service running on port 5002. 2. Seat available.3. User logged in.	Step: Send POST request to /api/bookings with X-User-Email. Input:Header: X-User-Email: user@example.com{ "showtime_id": "1", "seat_number": "A1" }	Status: 201 CreatedResponse: booking_id and details. Post: Booking created; Seat marked as booked.
TC_BOOKING_02	Verify booking multiple seats	1. Booking Service running. 2. Seats available. 3. User logged in.	Step: Send POST request to /api/bookings with seat_numbers array. Input:Header: X-User-Email: user@example.com{ "showtime_id": "1", "seat_numbers": ["A1", "A2", "A3"] }	Status: 201 CreatedResponse: booking_id and list of booked seats. Post: Booking created; All seats marked as booked.
TC_BOOKING_03	Verify booking without authorization	1. Booking Service running.	Step: Send POST request to /api/bookings	Status: 401 UnauthorizedResponse: {"error":

			without X-User-Email. Input: { "showtime_id": "1", "seat_number": "A1" }	"Unauthorized: Access via Gateway"} Post: No booking created.
TC_BOOKING_04	Verify retrieving user bookings	1. Booking Service running. 2. User has bookings.	Step: Send GET request to /api/bookings/my . Input: Header X-User-Email: user@example.co m.	Status: 200 OKResponse: List of user's bookings. Post: Database unchanged.
TC_BOOKING_05	Verify retrieving booking details	1. Booking Service running. 2. booking_id exists.	Step: Send GET request to /api/bookings/{b ooking_id}. Input: URL param booking_id.	Status: 200 OKResponse: Detailed booking info. Post: Database unchanged.
TC_BOOKING_06	Verify updating booking status	1. Booking Service running. 2. booking_id exists.	Step: Send PUT request to /api/bookings/{b ooking_id}/status . Input: { "status": "confirmed" }	Status: 200 OKResponse: {"message": "Status updated"} Post: Status updated in DB.
TC_BOOKING_07	Verify deleting/canceling booking	1. Booking Service running. 2. booking_id exists.	Step: Send DELETE request to /api/bookings/{b ooking_id}.	Status: 200 OKResponse: {"message": "Deleted"} Post: Booking

			Input: URL param booking_id.	removed; Seat becomes available.
TC_BOOKING_08	Verify retrieving seats for showtime	1. Booking Service running. 2. showtime_id exists.	Step: Send GET request to /api/showtimes/{ showtime_id}/se ats. Input: URL param showtime_id.	Status: 200 OKResponse: List of seats with status (available/booked) Post: Database unchanged.

5. PAYMENT SERVICE

Test Case ID	Test Objective	Pre-condition	Test Steps & Input Data	Expected Result & Post-condition
TC_PAYMENT_01	Verify successful payment	1. Payment Service running on port 5003. 2. Booking exists and unpaid.	Step: Send POST request to /api/payments. Input: { "booking_id": "1" } }	Status: 201 CreatedResponse: {"message": "Payment successful", "status": "PAID"} Post: Booking confirmed; Invoice event sent to RabbitMQ.
TC_PAYMENT_02	Verify payment for non-existent booking	1. Payment Service running.	Step: Send POST request to /api/payments with invalid ID. Input: { "booking_id": "99999" } }	Status: 404 Not FoundResponse: {"error": "Booking not found"} Post: No change in system.
TC_PAYMENT_03	Verify payment when Booking Service is down	1. Payment Service running. 2. Booking Service unavailable.	Step: Send POST request to /api/payments. Input: { "booking_id": "1" } }	Status: 503 Service UnavailableResponse: {"error": "Booking Service is unreachable"} Post: No change in system.
TC_PAYMENT_04	Verify adding payment method	1. Payment Service running. 2. User logged in.	Step: Send POST request to /api/payment-methods. Input: Header: X-User-Email: user@example.co	Status: 201 CreatedResponse: Masked card info **** * 5678. Post: Card saved with masked

			<pre>m{ "card_number": "1234567812345 678", "card_holder": "NGUYEN VAN A" }</pre>	number.
TC_PAYMENT_05	Verify retrieving payment methods	1. Payment Service running. 2. User has saved cards.	Step: Send GET request to /api/payment-methods. Input: Header X-User-Email: user@example.com.	Status: 200 OK Response: List of saved cards (masked). Post: Database unchanged.
TC_PAYMENT_06	Verify deleting payment method	1. Payment Service running. 2. card_id exists.	Step: Send DELETE request to /api/payment-methods/{card_id}. Input: Header X-User-EmailURL param card_id.	Status: 200 OK Response: {"message": "Card deleted"} Post: Card removed from DB.
TC_PAYMENT_07	Verify updating payment method	1. Payment Service running. 2. card_id exists.	Step: Send PUT request to /api/payment-methods/{card_id}. Input: Header X-User-Email { "card_holder": "NGUYEN VAN B" }	Status: 200 OK Response: {"message": "Card updated"} Post: Card holder name updated in DB.